```
# Install TensorFlow Datasets
!pip install tensorflow-datasets
```

```
Requirement already satisfied: tensorflow-datasets in /usr/local/lib/python3.12/dist-packages (4.9.9)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (1.4.0)
Requirement already satisfied: array_record>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow-datase
Requirement already satisfied: dm-tree in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.1.10)
Requirement already satisfied: etils>=1.9.1 in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,
Requirement already satisfied: immutabledict in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (4
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.0.2)
Requirement already satisfied: promise in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.3)
Requirement already satisfied: protobuf>=3.20 in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (5.9.5)
Requirement already satisfied: pyarrow in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (18.1.0)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets)
Requirement already satisfied: simple_parsing in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (
Requirement already satisfied: tensorflow-metadata in /usr/local/lib/python3.12/dist-packages (from tensorflow-datase
Requirement already satisfied: termcolor in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (3.3.0
Requirement already satisfied: toml in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.10.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (4.67.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.1.2)
Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree]
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree]
Requirement already satisfied: typing_extensions in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath
Requirement already satisfied: zipp in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree]>=
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->tensor
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->
Requirement already satisfied: attrs>=18.2.0 in /usr/local/lib/python3.12/dist-packages (from dm-tree->tensorflow-dat
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from promise->tensorflow-datasets) (1.
Requirement already satisfied: docstring-parser~=0.15 in /usr/local/lib/python3.12/dist-packages (from simple_parsing
Requirement already satisfied: googleapis-common-protos<2,>=1.56.4 in /usr/local/lib/python3.12/dist-packages (from t
```

```
import numpy as np
import tensorflow_datasets as tfds

# Load EMNIST letters dataset
(train_ds, test_ds), ds_info = tfds.load(
    'emnist/letters',
    split=['train', 'test'],
    as_supervised=True,
    with_info=True
)

# Convert train dataset
for images, labels in train_ds.batch(124800).take(1):
    X_train_letter = images.numpy()
    y_train_letter = labels.numpy()

# Convert test dataset
for images, labels in test_ds.batch(20800).take(1):
    X_test_letter = images.numpy()
    y_test_letter = labels.numpy()

print("Training:", X_train_letter.shape)
print("Testing:", X_test_letter.shape)
```

```
WARNING:absl:Variant folder /root/tensorflow_datasets/emnist/letters/3.1.0 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size)
Dl Completed...: 100% 1/1 [00:30<00:00, 16.60s/ url]
Dl Size...: 100% 535/535 [00:30<00:00, 29.85 MiB/s]
Extraction completed...: 100% 30/30 [00:30<00:00, 30.76s/ file]
Extraction completed...: 100% 4/4 [00:01<00:00, 3.45 file/s]
Dataset emnist downloaded and prepared to /root/tensorflow_datasets/emnist/letters/3.1.0. Subsequent calls will reuse
Training: (88800, 28, 28, 1)
Testing: (14800, 28, 28, 1)
```

```
X_train_letter
```

```
array([[[[0],
         [0],
```

```
            [0],
            ...,
            [0],
            [0],
            [0]],

           [[0],
            [0],
            [0],
            ...,
            [0],
            [0],
            [0]],

           [[0],
            [0],
            [0],
            ...,
            [0],
            [0],
            [0]],

           ...,

           [[0],
            [0],
            [0],
            ...,
            [0],
            [0],
            [0]],

           [[0],
            [0],
            [0],
            ...,
            [0],
            [0],
            [0]],

           [[0],
            [0],
            [0],
            ...,
            [0],
            [0],
            [0]]],


          [[[0],
            [0],
            [0],
            ...,
            [0],
            [0],
```

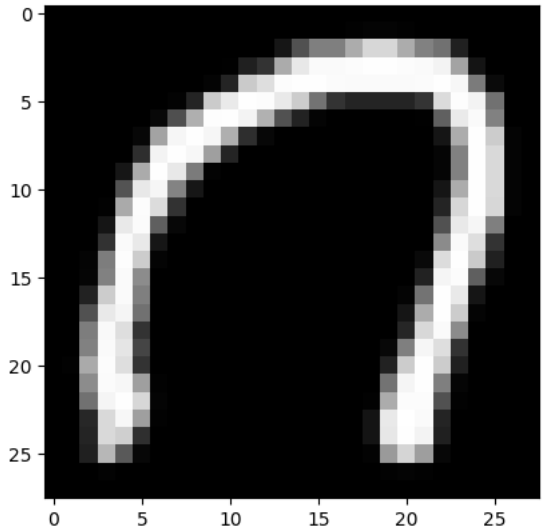
```
y_train_letter
```

```
array([25,  7, 25, ..., 18, 17, 13])
```

```
print(X_train_letter.shape)
print(y_train_letter.shape)

print(X_test_letter.shape)
print(y_test_letter.shape)
```

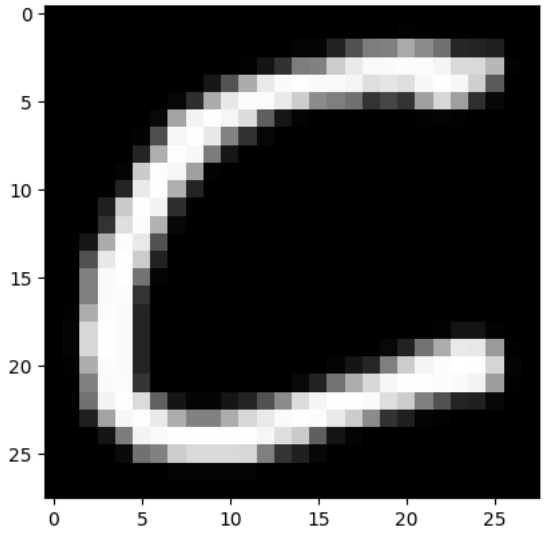
```
(88800, 28, 28, 1)
(88800,)
(14800, 28, 28, 1)
(14800,)
```

```
import matplotlib.pyplot as plt
import numpy as np

# Find image whose label is C (3)
index = np.where(y_train_letter == 3)[0][0]

plt.imshow(X_train_letter[index].squeeze(), cmap='gray')
plt.show()

print("Alphabet = C")
```




```
Alphabet = C
```

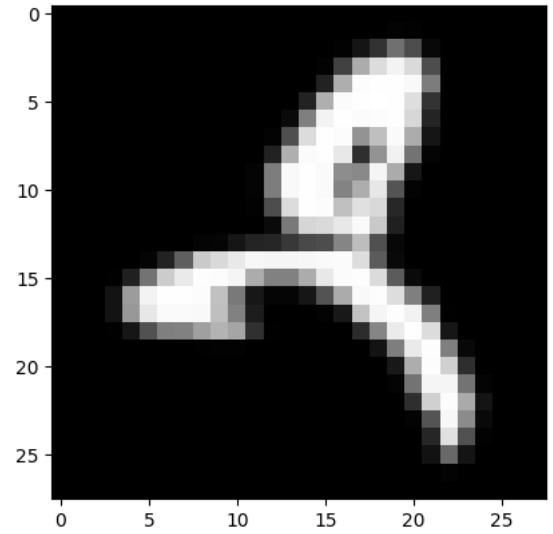
```
import matplotlib.pyplot as plt
import numpy as np

img = np.rot90(X_train_letter[index].squeeze(), -1)
img = np.fliplr(img)

plt.imshow(img, cmap='gray')
plt.show()

print("Alphabet = C")
```




```
Alphabet = C
```

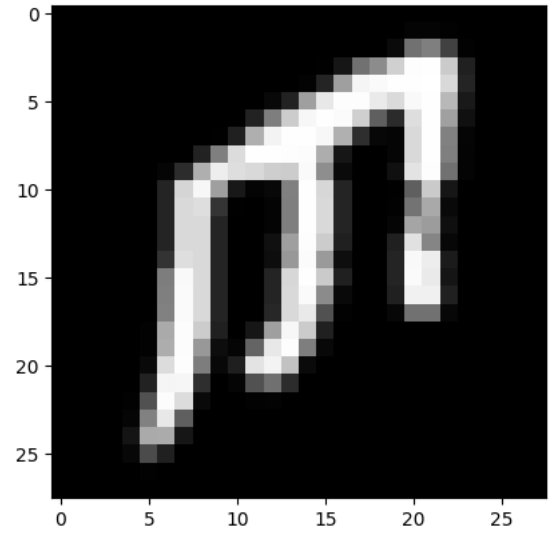
```
import matplotlib.pyplot as plt
import numpy as np

# Find image whose label is D (4)
index = np.where(y_train_letter == 4)[0][0]
```

```
plt.imshow(X_train_letter[index].squeeze(), cmap='gray')
plt.show()

print("Alphabet = D")
```



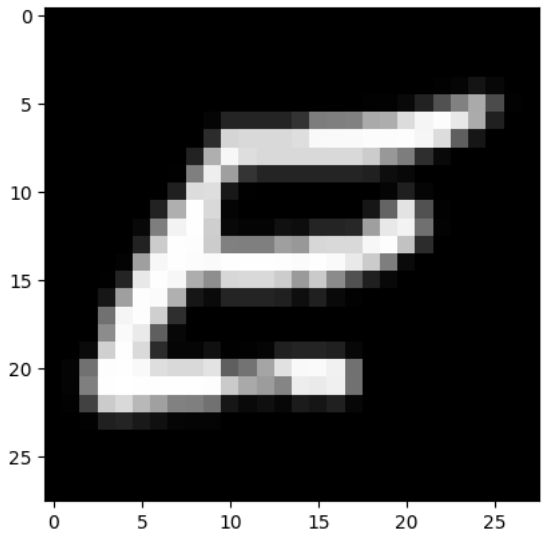
```
Alphabet = D
```

```
import matplotlib.pyplot as plt
import numpy as np

# Find image whose label is E (5)
index = np.where(y_train_letter == 5)[0][0]

plt.imshow(X_train_letter[index].squeeze(), cmap='gray')
plt.show()

print("Alphabet = E")
```



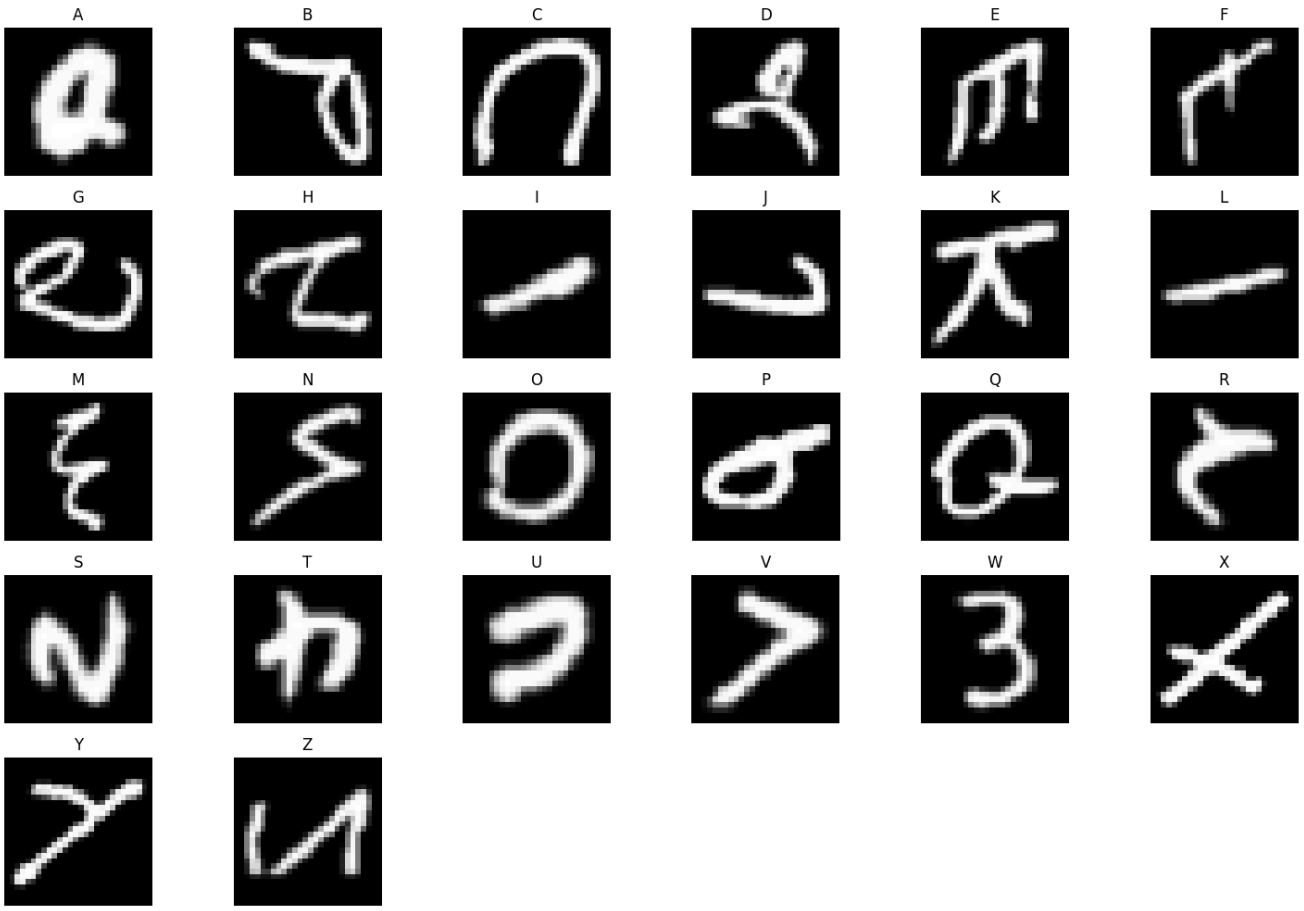
```
Alphabet = E
```

```
import matplotlib.pyplot as plt
import numpy as np

img = np.rot90(X_train_letter[index].squeeze(), -1)
img = np.fliplr(img)

plt.imshow(img, cmap='gray')
plt.show()

print("Alphabet = E")
```




```
Alphabet = E
```

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(15,10))

for letter in range(1,27):   # A=1 to Z=26

    # Find first occurrence of each alphabet
    index = np.where(y_train_letter == letter)[0][0]

    plt.subplot(5,6,letter)
    plt.imshow(X_train_letter[index].squeeze(), cmap='gray')
    plt.title(chr(letter + 64))
    plt.axis('off')

plt.tight_layout()
plt.show()
```




```
print(X_train_letter.shape)
print(y_train_letter.shape)

print(X_test_letter.shape)
print(y_test_letter.shape)
```

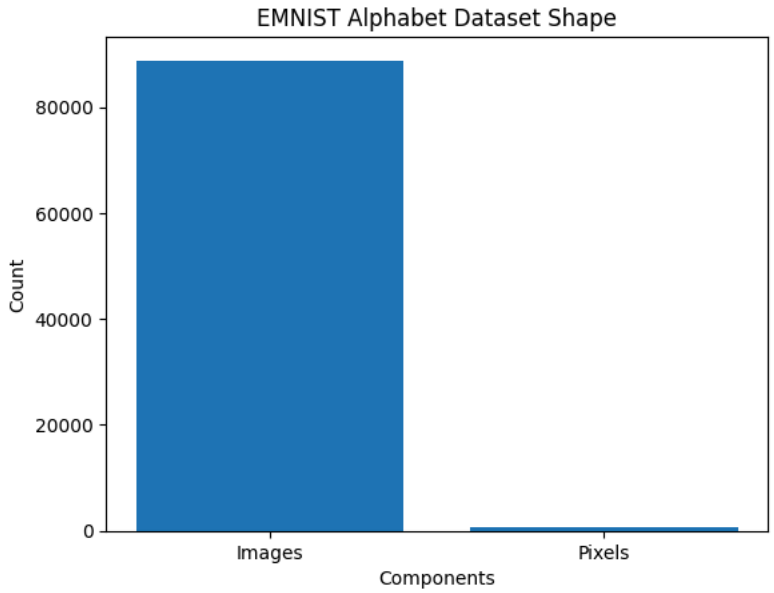
```
(88800, 28, 28, 1)
(88800,)
(14800, 28, 28, 1)
(14800,)
```

```
X_train_letter = X_train_letter.reshape(88800, 784)
X_test_letter = X_test_letter.reshape(14800, 784)
```

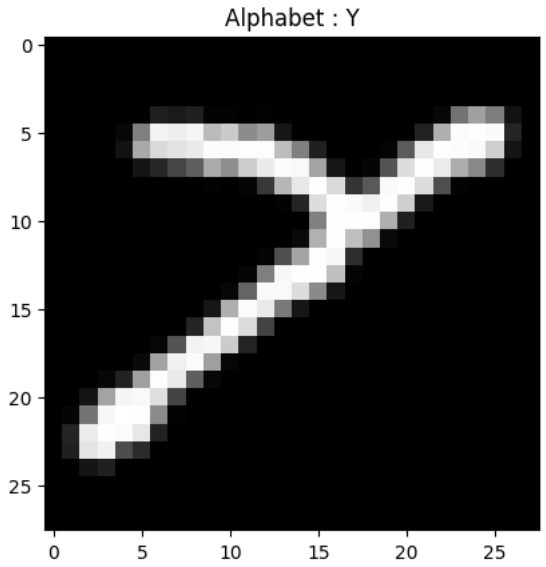
```
X_train_letter.shape
```

```
(88800, 784)
```

```
import matplotlib.pyplot as plt

# Data
dimensions = ['Images', 'Pixels']
values = [88800, 784]

# Graph
plt.bar(dimensions, values)

plt.title("EMNIST Alphabet Dataset Shape")
plt.xlabel("Components")
plt.ylabel("Count")

plt.show()
```




```
import matplotlib.pyplot as plt

plt.imshow(X_train_letter[0].reshape(28,28), cmap='gray')

alphabet = chr(y_train_letter[0] + 64)

plt.title("Alphabet : " + alphabet)

plt.show()
```




```
X_train_letter
```

```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], dtype=uint8)
```

```
X_test_letter
```

```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
```

```
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]], dtype=uint8)
```

```
print(np.unique(X_train_letter[0]))
```

```
[  0   1   2   3   4   5   6   7   9  18  20  21  22  24  26  32  33  34
  35  36  37  39  43  46  52  57  67  69  76  78  79  84  89  91  94  95
 100 115 122 125 127 128 129 137 138 139 140 142 145 157 159 160 162 163
 165 169 170 173 175 177 183 186 189 191 196 202 203 204 206 207 217 218
 220 221 222 228 229 231 232 234 235 236 238 240 241 243 245 246 247 248
 249 250 251 252 253 254 255]
```

... hides many values in the middle

```
y_train_letter
```

```
array([25,  7, 25, ..., 18, 17, 13])
```

```
y_test_letter
```

```
array([13,  8,  1, ..., 12, 17,  6])
```

```
y_train_letter.shape
```

```
(88800,)
```

```
from tensorflow.keras.utils import to_categorical

y_train_letter = to_categorical(y_train_letter, num_classes=27)
y_test_letter = to_categorical(y_test_letter, num_classes=27)
```

```
y_train_letter[1]
```

```
array([0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
       0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
y_test_letter[1]
```

```
array([0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0.,
       0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
import tensorflow as tf

# Creating base neural network
model = tf.keras.Sequential([

    # First Hidden Layer
    tf.keras.layers.Dense(256, activation='relu', input_shape=(784,)),

    # Second Hidden Layer
    tf.keras.layers.Dense(64, activation='relu'),

    # Third Hidden Layer
    tf.keras.layers.Dense(64, activation='relu'),

    # Output Layer for A–Z
    tf.keras.layers.Dense(27, activation='softmax')

])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	200,960
dense_1 (Dense)	(None, 64)	16,448
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 27)	1,755

```
 Total params: 223,323 (872.36 KB)
 Trainable params: 223,323 (872.36 KB)
```

```
# Output layer
model.layers[-1]
```

```
<Dense name=dense_3, built=True>
```

```
# Compiling the model
model.compile(
    loss='categorical_crossentropy',   # Loss Function
    optimizer='adam',                  # Optimizer
    metrics=['accuracy']               # Evaluation Metric
)
```

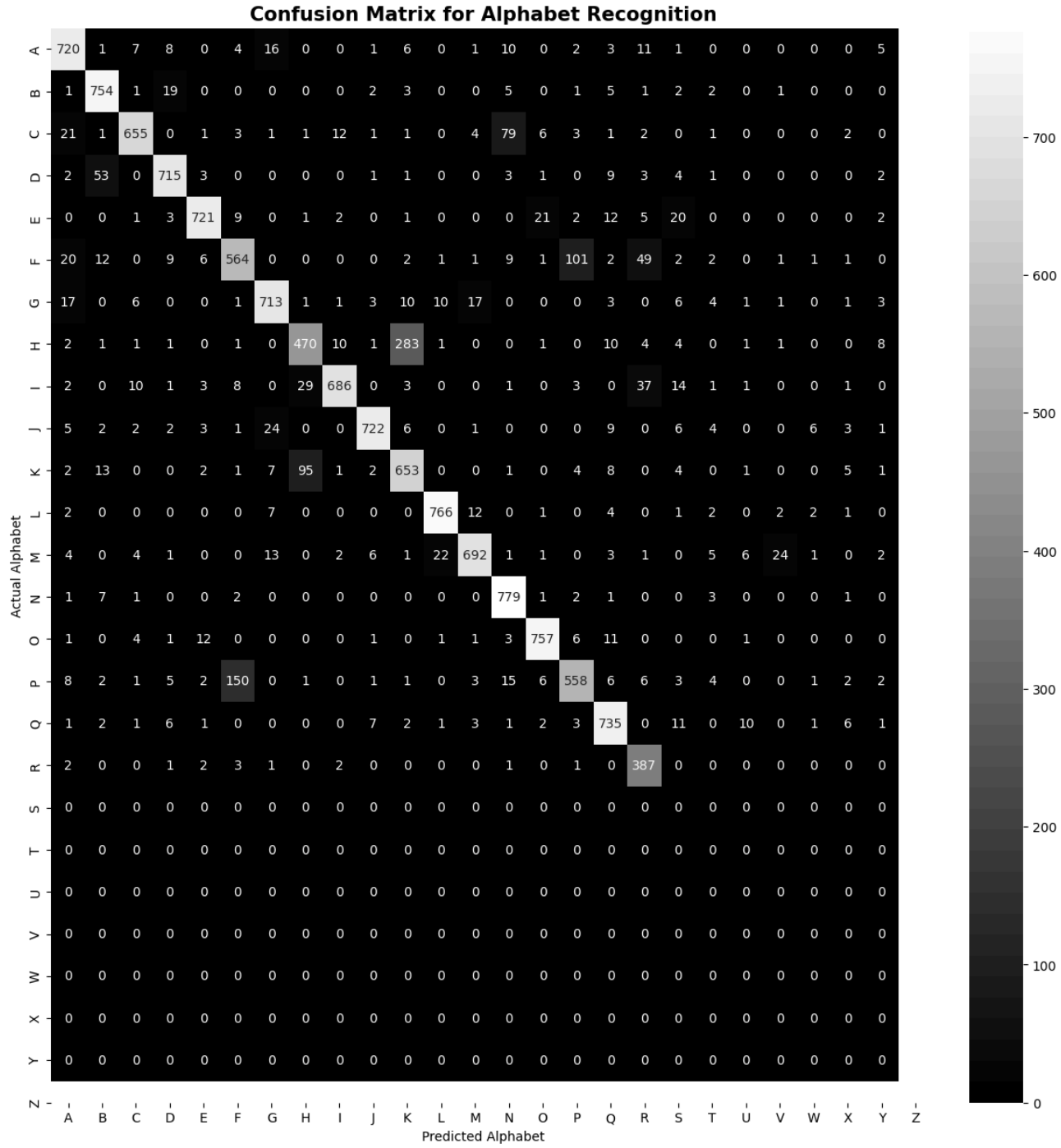
```
# Training the model

history = model.fit(
    X_train_letter,
    y_train_letter,
    batch_size=100,
    epochs=10,
    validation_data=(X_test_letter, y_test_letter)
)
```

```
Epoch 1/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21s 20ms/step - accuracy: 0.3643 - loss: 2.9746 - val_accuracy: 0.5419 - va
Epoch 2/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.6775 - loss: 1.1077 - val_accuracy: 0.7168 - val
Epoch 3/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.7859 - loss: 0.7032 - val_accuracy: 0.7819 - val_
Epoch 4/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.8285 - loss: 0.5543 - val_accuracy: 0.8061 - val_
Epoch 5/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8495 - loss: 0.4799 - val_accuracy: 0.8234 - val_
Epoch 6/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9s 7ms/step - accuracy: 0.8650 - loss: 0.4281 - val_accuracy: 0.8205 - val_
Epoch 7/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8762 - loss: 0.3923 - val_accuracy: 0.8443 - val_
Epoch 8/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - accuracy: 0.8831 - loss: 0.3643 - val_accuracy: 0.8430 - val_
Epoch 9/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8900 - loss: 0.3400 - val_accuracy: 0.8336 - val_
Epoch 10/10
888/888 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.8943 - loss: 0.3196 - val_accuracy: 0.8580 - val_
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Convert one-hot encoded labels back to normal labels
y_test_letter_eval = np.argmax(y_test_letter, axis=1)

# Predict test images
y_predicts = model.predict(X_test_letter)
y_predicts = np.argmax(y_predicts, axis=1)

# Confusion Matrix
con_mat = confusion_matrix(y_test_letter_eval, y_predicts)

# Alphabet labels
labels = [chr(i+64) for i in range(1,27)]

plt.figure(figsize=(15,15))
```

```
sns.heatmap(
    con_mat[1:,1:],     # ignore class 0
    annot=True,
    fmt='d',
    cmap='gray',
    xticklabels=labels,
    yticklabels=labels
)

plt.title(
    "Confusion Matrix for Alphabet Recognition",
    fontweight='bold',
    fontsize=15
)

plt.xlabel("Predicted Alphabet")
plt.ylabel("Actual Alphabet")

plt.show()
```

```
463/463 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```




```
from sklearn.metrics import classification_report
import numpy as np

# Convert one-hot encoded labels back to normal labels
y_test_letter_eval = np.argmax(y_test_letter, axis=1)

# Predict test images
y_predicts = model.predict(X_test_letter)
y_predicts = np.argmax(y_predicts, axis=1)

# Alphabet labels A–Z
```

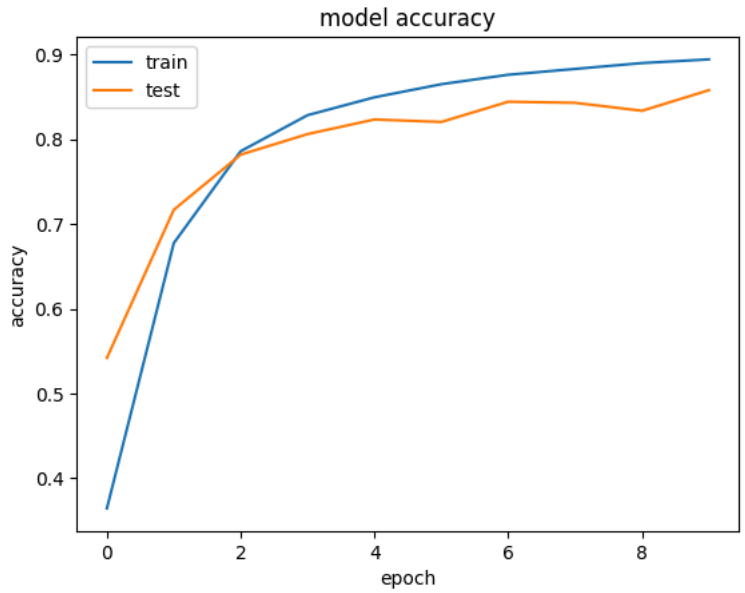
```
alphabet_labels = [chr(i+64) for i in range(1,27)]

# Generate report
print(
    classification_report(
        y_test_letter_eval,
        y_predicts,
        labels=range(1,27),
        target_names=alphabet_labels
    )
)
```

```
463/463 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
              precision    recall  f1-score   support

           A       0.89      0.81      0.85       800
           B       0.87      0.90      0.88       800
           C       0.88      0.94      0.91       800
           D       0.93      0.82      0.87       800
           E       0.92      0.89      0.91       800
           F       0.95      0.90      0.93       800
           G       0.75      0.70      0.72       800
           H       0.90      0.89      0.90       800
           I       0.79      0.59      0.67       800
           J       0.96      0.86      0.91       800
           K       0.96      0.90      0.93       800
           L       0.67      0.82      0.74       800
           M       0.95      0.96      0.96       800
           N       0.93      0.86      0.89       800
           O       0.84      0.97      0.90       800
           P       0.94      0.95      0.95       800
           Q       0.78      0.70      0.74       800
           R       0.89      0.92      0.90       800
           S       0.76      0.97      0.85       400
           T       0.00      0.00      0.00         0
           U       0.00      0.00      0.00         0
           V       0.00      0.00      0.00         0
           W       0.00      0.00      0.00         0
           X       0.00      0.00      0.00         0
           Y       0.00      0.00      0.00         0
           Z       0.00      0.00      0.00         0

    accuracy                           0.86     14800
   macro avg       0.64      0.63      0.63     14800
weighted avg       0.87      0.86      0.86     14800

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is il
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is il
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is il
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

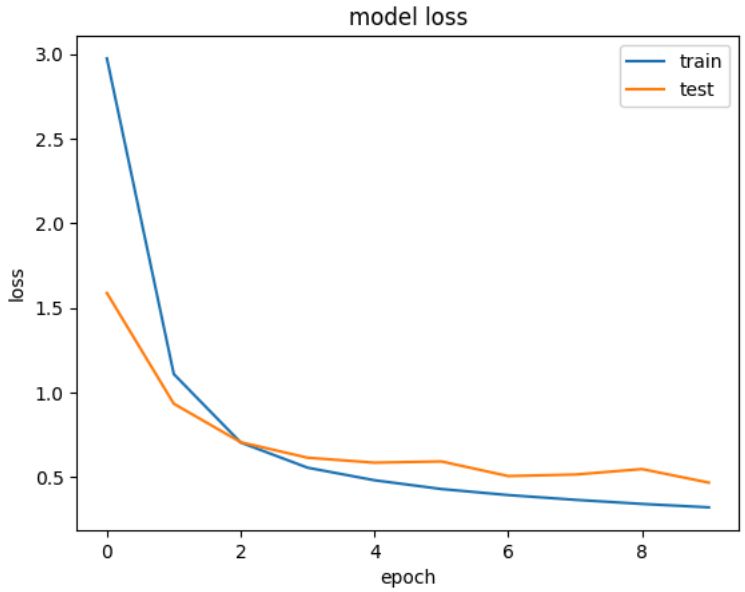
```
print(history.history.keys())
```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

```
# summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='best')
plt.show()
```



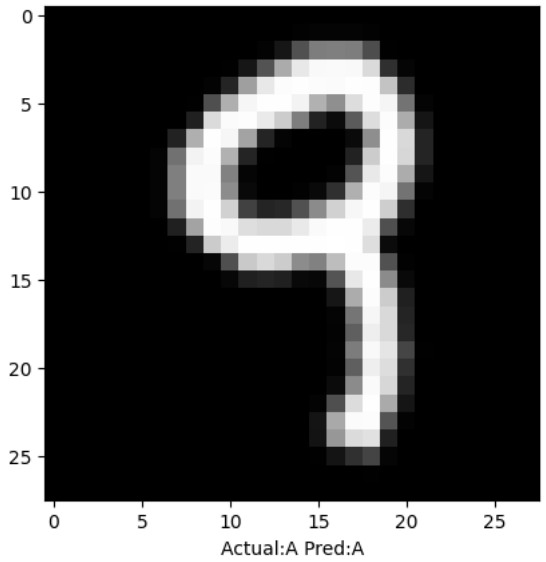

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='best')
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Find alphabet A (A = 1)
i = np.where(np.argmax(y_test_letter, axis=1) == 1)[0][0]

# Predict single image
y_predict_single = model.predict(X_test_letter[[i]])
y_predict_single = np.argmax(y_predict_single, axis=1)

# Convert labels to alphabets
actual = chr(np.argmax(y_test_letter[i]) + 64)
predicted = chr(y_predict_single[0] + 64)

# Display image
plt.imshow(X_test_letter[i].reshape(28,28), cmap='gray')

plt.xlabel(
```

```
    "Actual:{} Pred:{}".format(
        actual,
        predicted
    )
)

plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
```




```
# Rotate and flip the image before displaying:

import numpy as np
import matplotlib.pyplot as plt

img = X_test_letter[i].reshape(28,28)

# Correct orientation
img = np.rot90(img, -1)
img = np.fliplr(img)

# Display image
plt.imshow(X_test_letter[i].reshape(28,28), cmap='gray')

plt.xlabel(
    "Actual:{} Pred:{}".format(
        actual,
        predicted
    )
)

plt.imshow(img, cmap='gray')
plt.show()
```